CREACIÓN Y USO DE LIBRERÍAS

Imagine que usted estuvo trabajando en un proyecto para una empresa de transporte de artículos en el cual se requería almacenar los datos sobre los empleados que manejan los vehículos, los artículos que transportan y las fechas de salida y llegada de los vehículos. Para este proyecto usted implementó los TDAs Empleado, Vehículo, Artículo y Fecha. En está diseñando un nuevo sistema para el manejo de un supermercado, y se da cuenta que los TDAs Empleado, Artículo y Fecha le pueden servir.

Una forma de reusar los TDAs es copiar todo el código de los mismos en el archivo del programa a desarrollar, sin embargo esta aproximación no es correcta por diferentes razones:

- Se realizan copias de un mismo código.
- El tamaño del archivo es grande.
- La falta de modularidad.
- Las mejoras hechas a un TDA no se reflejan en todos los proyectos.
- El mantenimiento del código es complicado.

Por esta razón, es una buena práctica crear librerías que implementen los TDAs para luego incluirlas en aquellos proyectos que lo requieran. Cada una de estas librerías puede ser mantenida por separado sin que exista la necesidad de modificar el resto de la aplicación.

Creación de una Librería

En lenguaje C, una librería está formada por dos archivos:

1. Un archivo de extensión ".h", llamado archivo de cabecera que contiene:
   1.1. La estructura de datos que representa al TDA.
   1.2. La declaración de las operaciones del TDA.

2. Un archivo de extensión ".c" que contiene la definición de todas las operaciones declaradas en el archivo ".h".

La estructura de un archivo de cabecera es la siguiente:

```
#ifndef NOMBRE_H
#define NOMBRE_H

// Tipo
// Declaración de las operaciones

#endif
```

Las directivas #ifndef y #endif son directivas del preprocesador al igual que las directivas ya conocidas #include y #define (las cuales permiten incluir un archivo de cabecera en otro archivo, y definir símbolos respectivamente).

El bloque #ifndef – #endif se utiliza para realizar una compilación condicional: #ifndef verifica si no está definido un identificador mediante la directiva #define, por ejemplo NOMBRE_H, y en caso de que no esté definido, permite compilar el código que se encuentre entre el #ifndef y el #endif, directiva que finaliza el bloque.

Esto es necesario para evitar que se incluya un mismo archivo de cabecera en múltiples oportunidades, por ejemplo, si existe una librería “a.h” y una librería “b.h” que incluye a la anterior, al hacer un programa que incluya ambas librerías, se estaría incluyendo dos veces “a.h” sin estar al tanto de ello.

Ejemplo:

Se tiene el TDA Punto2D, que representa un punto bidimensional en el espacio. Cuenta con la siguiente especificación operacional:

Sintaxis

TDA Punto2D

origen:		$\rightarrow$	Punto2D
esOrigen:	Punto2D	$\rightarrow$	lógico
obtenerX:	Punto2D	$\rightarrow$	real
obtenerY:	Punto2D	$\rightarrow$	real
modificarX:	Punto2D $\times$ real	$\rightarrow$	Punto2D
modificarY:	Punto2D $\times$ real	$\rightarrow$	Punto2D
transladar:	Punto2D $\times$ real $\times$ real	$\rightarrow$	Punto2D

Semántica

Sean p, p’ $\in$ Punto2D y x, y, a, b, tx, ty $\in$ R. Se tomará en cuenta que p’ es el resultado de cambiar p.

{ Pre: }
func origen(): Punto2D
{ Post: origen $\leftarrow$ (0, 0) }

{ Pre: }
func esOrigen(Punto2D: p): **lógico**
{ Post: esOrigen ← (p = (0, 0)) }

{ Pre: }
func obtenerX(Punto2D: p): **real**
{ Post: Si p = (x, y) entonces obtenerX ← x }

{ Pre: }
func obtenerY(Punto2D: p): **real**
{ Post: Si p = (x, y) entonces obtenerY ← y }

{ Pre: }
proc modificarX(**ref** Punto2D: p; **real**: a)
{ Post: Si p = (x, y) entonces p' = (a, y) }

{ Pre: }
proc modificarX(**ref** Punto2D: p; **real**: b)
{ Post: Si p = (x, y) entonces p' = (x, b) }

{ Pre: }
proc transladar(**ref** Punto2D: p; **real**: tx, ty)
{ Post: Si p = (x, y) entonces p' = (x + tx, y + ty) }

Implementación

"Punto2D.h":

```
#ifndef PUNTO2D_H
#define PUNTO2D_H

typedef struct
{
   float x, y;
}Punto2D;

Punto2D origen();
int esOrigen(Punto2D p);
float obtenerX(Punto2D p);
float obtenerY(Punto2D p);
void modificarX(Punto2D *p, float a);
void modificarY(Punto2D *p, float b);
void transladar(Punto2D *p, float tx, float ty);

#endif
```

"Punto2D.c":

```c
#include <stdio.h>
#include "Punto2D.h"

Punto2D origen()
{
   Punto2D p;
   p.x = 0.0;
   p.y = 0.0;
   return(p);
}

int esOrigen(Punto2D p)
{
   return(p.x == 0.0 && p.y == 0.0);
}

float obtenerX(Punto2D p)
{
   return(p.x);
}

float obtenerY(Punto2D p)
{
   return(p.y);
}

void modificarX(Punto2D *p, float a)
{
   p->x = a;
}

void modificarY(Punto2D *p, float b)
{
   p->y = b;
}

void transladar(Punto2D *p, float tx, float ty)
{
   p->x = p->x + tx;
   p->y = p->y + ty;
}
```

Uso de una Librería

Para usar la librería se debe incluir como archivo de cabecera en el programa que se esté implementando. Luego se pueden declarar las variables del TDA y usar las operaciones del mismo con normalidad.

Ejemplo:

“Principal.c”

```
#include <stdio.h>
#include "Punto2D.h"

int main()
{
	Punto2D p;
	p = origen();
	modificarX(&p, 1.0);
	modificarY(&p, 2.5);
	printf("(%f, %f)", obtenerX(p), obtenerY(p));
	return(0);
}
```

Para compilarlo, se debe compilar primero la librería de la siguiente forma:

> gcc -c Punto2D.c

Esto genera un archivo objeto llamado “Punto2D.o”, el cual debe ser enlazado con el programa principal. Para compilar el programa “Principal.c” se coloca:

> gcc Punto2D.o Principal.c -o Principal

Ahora bien, ¿qué sucedería si en vez de tener un proyecto que haga uso de una sola librería, se tuviera un proyecto que use 5 librerías?

El proceso de compilación sería algo así:

> gcc -c lib1.c
> gcc -c lib2.c
> gcc -c lib3.c
> gcc -c lib4.c
> gcc -c lib5.c
> gcc lib1.o lib2.o lib3.o lib4.o lib5.o Principal.c -o Principal

Si este proceso se tuviera que realizar en múltiples oportunidades resultaría sumamente tedioso, por esta razón existe un archivo llamado makefile que permite guardar todas las reglas de compilación para así, con sólo una llamada, poder ejecutar todos los comandos.

Esto simplifica la compilación de proyectos complejos y minimiza los errores de dependencias.

Makefile

Un makefile es un archivo formado por una serie de reglas necesarias para realizar la compilación de un programa, y además puede contener variables y comentarios.

Cada regla tiene la siguiente forma:

```
objetivo: dependencias
     comandos
```

El objetivo representa el archivo que se va a obtener después de aplicar los comandos sobre las dependencias. Las dependencias son todos aquellos archivos necesarios para poder crear el archivo objetivo.

Nótese que antes de un comando debe haber, obligatoriamente, un tabulador.

Las variables se declaran como NOMBRE = dato, y se obtiene su valor colocándolas entre paréntesis y con un carácter $ al inicio de la siguiente manera: $(NOMBRE).

Los comentarios comienzan con un caracter de numeral (#), y hacen que se ignore el resto de la línea.

Ejemplo:

```
# Makefile de Punto2D

# Variables
CC = gcc
EJECUTABLE = Principal

# Programa Principal
all: Punto2D.o
   $(CC) Punto2D.o Principal.c -o $(EJECUTABLE)

# Librerías
Punto2D.o: Punto2D.h Punto2D.c
   $(CC) -c Punto2D.c

# Borrar los Archivos Objeto y el Ejecutable
clean:
   rm -rf *.o $(EJECUTABLE)
```

Para compilar el programa “Principal.c” se coloca el comando:
> make

Para borrar los archivos objeto y el ejecutable se coloca el comando:
> make clean